AWS SECRETS MANAGER — STEP-BY-STEP SETUP GUIDE

True Zero-Downtime
RDS MySQL Credential Rotation

Dual-Secret • Staggered Cron • Intelligent Application Failover

Primary + Backup	2-minute offset	< 100 ms	0 proven
Dual Secrets	Stagger Gap	Failover	Hard Failures

Rotation Engine	App Layer	Schedule
Standard AWS Lambda (unmodified)	**Python — SQLAlchemy or pymysql**	**cron(0/5) + cron(2/5) staggered**

Overview

Standard AWS Secrets Manager rotation revokes the active DB user session during the setSecret step. Any in-flight query or open transaction at that exact millisecond receives an OperationalError and is lost. At scale — with 50,000 concurrent threads — this creates a cascade of failures every time rotation fires.

This guide builds a system that makes credential rotation invisible to the application by combining two complementary strategies:

Layer	Strategy	What It Does
Infrastructure	Dual-secret staggered cron	Two independent secrets on offset schedules — one is always stable while the other rotates
Application	Dual-engine failover manager	Detects dropped connections instantly, retries on the backup engine, rehydrates primary in background

How the Stagger Works

```
Primary secret rotates at:  0, 5, 10, 15, 20... minutes  (cron 0/5)
Backup  secret rotates at:  2, 7, 12, 17, 22... minutes  (cron 2/5)

The 2-minute gap guarantees one secret is always fully stable while the other rotates.
The application detects primary failures and transparently uses backup during the gap.
```

Document Structure

Phase	Steps	What You Build
Phase 1	1.1 — 1.4	VPC, security group, VPC endpoints, RDS, database users, dual secrets
Phase 2	2.1 — 2.3	Rotation Lambda attached to both secrets, self-healing code modification
Phase 3	3.1 — 3.2	Backup trigger Lambda (deploy_backup_trigger.py) + two EventBridge rules
Phase 4	4.1 — 4.3	Application failover layer — SQLAlchemy vs pymysql comparison + implementation
Phase 5	5.1 — 5.2	Stress test (transaction_stress_test.py) and pass criteria

Variables — Fill In Before Starting

Variable	Description	Your Value
<YOUR_REGION>	AWS region (e.g. us-west-2)	
<DB_NAME>	RDS instance identifier	
<RDS_ENDPOINT>	RDS endpoint URL (after RDS creation)	
<MASTER_PASSWORD>	Admin master password	
<APP_USER_PASSWORD>	Initial app-user password	
<APP_BACKUP_PASSWORD>	Initial app-user-backup password	
<MASTER_SECRET_ARN>	ARN of db-rotation-master (after creation)	
<PRIMARY_SECRET_ARN>	ARN of db-rotation-new-secret	
<BACKUP_SECRET_ARN>	ARN of db-rotation-backup-secret	

PHASE 1

AWS Infrastructure Setup & Secrets Initialization

STEP 1.1 Create VPC and Network Layer

> Do NOT create a NAT Gateway. All AWS service traffic flows through VPC Interface Endpoints — cheaper, more secure, and required for Lambda in a private subnet.

Create VPC

1. VPC → Create VPC → select VPC and more
2. Configure:

Setting	Value	Note
Name	rotation-vpc	
IPv4 CIDR	10.0.0.0/16	Adjust if needed
Public subnets	0	Not needed
Private subnets	2	Across 2 AZs
NAT Gateway	None	Replaced by VPC Endpoints
Enable DNS hostnames	Checked	Required for private DNS on endpoints
Enable DNS resolution	Checked	Required for private DNS on endpoints

3. Click Create VPC

Create Security Group — rotation-sg

4. EC2 → Security Groups → Create security group
5. Name: rotation-sg, VPC: rotation-vpc
6. Inbound rules → Edit inbound rules → Add rule (twice):

Type	Port	Source	Purpose
HTTPS	443	rotation-sg (self)	Lambda → VPC Endpoints
MySQL/Aurora	3306	rotation-sg (self)	Lambda → RDS

7. Outbound rules → Edit outbound rules → delete default → Add rule (twice):

Type	Port	Destination	Purpose
HTTPS	443	rotation-sg (self)	Reach VPC Endpoints
MySQL/Aurora	3306	rotation-sg (self)	Reach RDS

8. Save rules

> Self-referencing rules allow all resources that share rotation-sg to communicate with each other automatically — Lambda, RDS, and all VPC Endpoints.

STEP 1.2 Create VPC Endpoints

> All 5 endpoints must be created and show Available before proceeding. Missing even one causes Lambda to silently timeout with zero log output.

Name	Service	Type
sm-endpoint	com.amazonaws.<YOUR_REGION>.secretsmanager	Interface
rds-endpoint	com.amazonaws.<YOUR_REGION>.rds	Interface
ssm-endpoint	com.amazonaws.<YOUR_REGION>.ssm	Interface
ssmmessages-endpoint	com.amazonaws.<YOUR_REGION>.ssmmessages	Interface
ec2messages-endpoint	com.amazonaws.<YOUR_REGION>.ec2messages	Interface

For each endpoint:

9. VPC → Endpoints → Create endpoint
10. Service category: AWS services → search and select from table above
11. VPC: rotation-vpc, Subnets: both private subnets, Security group: rotation-sg
12. Policy: Full access, Enable private DNS: Checked
13. Create endpoint — wait for Status: Available

> Enable private DNS must be checked on every Interface endpoint. Without it Lambda resolves AWS service URLs to public IPs which are unreachable from a private subnet.

STEP 1.3 Create RDS MySQL Instance

Create DB Subnet Group

14. RDS → Subnet groups → Create DB subnet group
15. Name: rotation-subnet-group, VPC: rotation-vpc, add both private subnets → Create

Create RDS Instance

16. RDS → Databases → Create database

Setting	Value
Engine	MySQL 8.0
DB instance identifier	<DB_NAME>
Master username	admin
Master password	<MASTER_PASSWORD>
VPC	rotation-vpc
Subnet group	rotation-subnet-group
Public access	No — CRITICAL
Security group	rotation-sg

17. Click Create database — wait 5-10 minutes for Status: Available
18. Go to Connectivity & security tab — copy the Endpoint URL → this is <RDS_ENDPOINT>

Create Database Users

Connect from a bastion EC2 in a public subnet (see previous guide for public subnet + bastion setup):

```
mysql -h <RDS_ENDPOINT> -u admin -p'<MASTER_PASSWORD>'
```

```
-- Primary rotating user
CREATE USER 'app-user'@'%' IDENTIFIED BY '<APP_USER_PASSWORD>';
GRANT ALL PRIVILEGES ON *.* TO 'app-user'@'%';

-- Backup user — independent user, independent password lifecycle
CREATE USER 'app-user-backup'@'%' IDENTIFIED BY '<APP_BACKUP_PASSWORD>';
GRANT ALL PRIVILEGES ON *.* TO 'app-user-backup'@'%';

FLUSH PRIVILEGES;
SELECT user, host FROM mysql.user WHERE user LIKE 'app%';
EXIT;
```

STEP 1.4 Create the Three Secrets

This system requires exactly three secrets. The architecture depends on keeping master, primary, and backup completely independent.

Secret Name	DB User	Rotates	Purpose
db-rotation-master	admin	Never	Master credentials — used by rotation Lambda to run SET PASSWORD
db-rotation-new-secret	app-user	Yes — primary schedule	Primary application credentials (cron 0/5)
db-rotation-backup-secret	app-user-backup	Yes — offset schedule	Backup application credentials (cron 2/5)

Create Master Secret

19. Secrets Manager → Store a new secret
20. Secret type: Credentials for Amazon RDS database
21. Username: admin, Password: <MASTER_PASSWORD>, Database: <DB_NAME>
22. Name: db-rotation-master → Store
23. Open the secret → copy the Secret ARN → this is <MASTER_SECRET_ARN>

Create Primary Application Secret

24. Store a new secret → Type: Credentials for Amazon RDS database
25. Username: app-user, Password: <APP_USER_PASSWORD>, Database: <DB_NAME>
26. Name: db-rotation-new-secret → Store
27. Open secret → Retrieve secret value → Edit → switch to Plaintext → replace with:

```
{
  "username": "app-user",
  "password": "<APP_USER_PASSWORD>",
  "engine": "mysql",
  "host": "<RDS_ENDPOINT>",
  "port": 3306,
  "dbname": "your_database_name",
  "dbInstanceIdentifier": "<DB_NAME>",
  "masterarn": "<MASTER_SECRET_ARN>"
}
```

28. Save — copy the Secret ARN → this is <PRIMARY_SECRET_ARN>

Create Backup Application Secret

29. Repeat the same steps — Username: app-user-backup, Password: <APP_BACKUP_PASSWORD>
30. Name: db-rotation-backup-secret — same JSON structure, change username and password fields
31. Save — copy the Secret ARN → this is <BACKUP_SECRET_ARN>

> The masterarn field in both application secrets must point to db-rotation-master. Without it the rotation Lambda cannot fetch admin credentials to run SET PASSWORD in MySQL.

PHASE 2

Rotation Lambda Setup

The same rotation Lambda is attached to both secrets. AWS handles the ALTER USER / SET PASSWORD commands. We make one modification to setSecret to enable self-healing — removing the AWSCURRENT credential validation that blocks rotation when credentials drift.

STEP 2.1 Enable Rotation on the Primary Secret

32. Open db-rotation-new-secret → Rotation configuration → Edit rotation
33. Fill in:

Field	Value
Automatic rotation	ON
Schedule expression	cron(0/5 * * * ? *) — fires at 0, 5, 10, 15... minutes
Rotation function	Create a new Lambda function
Lambda function name	SecretsManager-rotation-<DB_NAME>
Use a separate master secret	Checked
Master secret	db-rotation-master

34. Click Save — AWS creates the Lambda automatically

Attach rotation-sg to the Rotation Lambda

35. Lambda → SecretsManager-rotation-<DB_NAME> → Configuration → VPC → Edit
36. VPC: rotation-vpc, Subnets: both private subnets, Security groups: rotation-sg
37. Save — wait for Status: Active

STEP 2.2 Attach the Same Lambda to the Backup Secret

> We attach the identical Lambda to the backup secret. The Lambda reads SecretId from the event and handles whichever secret triggered it — no custom rotation logic needed.

38. Open db-rotation-backup-secret → Rotation configuration → Edit rotation
39. Automatic rotation: ON

40. Rotation function: Use existing Lambda function → select SecretsManager-rotation-<DB_NAME>
41. Use a separate master secret: Checked → db-rotation-master
42. Schedule expression: cron(2/5 * * * ? *) — fires at 2, 7, 12, 17... minutes
43. Click Save

> Both secrets now share one rotation Lambda. The staggered schedules are the only difference. No additional rotation code is required.

STEP 2.3 Apply the Self-Healing Modification to the Lambda Code

The default AWS setSecret function validates that the AWSCURRENT password works before proceeding. If credentials drift out of sync for any reason, this check causes rotation to fail permanently. We remove it so the master user can always force a password update.

How to Make the Change

44. Lambda → SecretsManager-rotation-<DB_NAME> → Code tab
45. Press Ctrl+F → search for: setSecret: Unable to log into database using current credentials
46. Find the block directly above this error — it looks like:

```
# FIND AND REMOVE this block from set_secret():
conn = get_connection(current_dict)
if not conn:
    logger.error('setSecret: Unable to log into database using current credentials for
secret %s' % arn)
    raise ValueError('Unable to log into database using current credentials for secret
%s' % arn)
conn.close()
```

47. Delete these 5 lines entirely
48. The function should flow directly from host validation to master credential fetch
49. Click Deploy

> After this change, setSecret skips AWSCURRENT validation and goes straight to logging in with master credentials to run SET PASSWORD. This makes rotation 100% resilient to credential sync failures — the master always has authority.

PHASE 3

Custom Staggered Schedule Pipeline

The rotation Lambda cannot be called directly by EventBridge because EventBridge sends a different event format than Secrets Manager. We need a lightweight trigger Lambda for the backup secret that cleans stuck AWSPENDING states and calls rotate_secret() with a fresh UUID token.

STEP 3.1 Create the Backup Trigger Lambda — deploy_backup_trigger.py

Create the Lambda Function

50. Lambda → Create function → Name: trigger-rotation-backup, Runtime: Python 3.10
51. Permissions: Create a new role with basic Lambda permissions → Create function

Add IAM Permission

52. Configuration → Permissions → click the Role name link
53. Add permissions → Create inline policy → JSON tab → paste:

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": [
      "secretsmanager:RotateSecret",
      "secretsmanager:CancelRotateSecret",
      "secretsmanager:DescribeSecret",
      "secretsmanager:UpdateSecretVersionStage"
    ],
    "Resource": "*"
  }]
}
```

54. Policy name: backup-trigger-policy → Create policy
55. Add permissions → Attach policies → AWSLambdaVPCAccessExecutionRole → Add

Attach VPC and rotation-sg

56. Configuration → VPC → Edit → rotation-vpc, both private subnets, rotation-sg → Save

Complete Code — deploy_backup_trigger.py

57. Code tab → select all → delete → paste the complete code below → Deploy

```python
# deploy_backup_trigger.py
# Backup Secret Trigger Lambda
# Invoked by EventBridge cron(2/5 * * * ? *) — fires at minutes 2, 7, 12, 17...
# Job: clean stuck AWSPENDING, then trigger rotation with a unique UUID token

import boto3
import logging
import uuid

logger = logging.getLogger()
logger.setLevel(logging.INFO)

# Update this value if your backup secret has a different name
BACKUP_SECRET_ID = 'db-rotation-backup-secret'


def lambda_handler(event, context):
    client = boto3.client('secretsmanager', region_name='<YOUR_REGION>')

    try:
        # ── 1. Remove stuck AWSPENDING stage ──────────────────────────────
        # A failed rotation leaves AWSPENDING on an old version, blocking
        # future rotations with 'A previous rotation is not complete'.
        # We strip it off before triggering the new rotation.
        metadata = client.describe_secret(SecretId=BACKUP_SECRET_ID)
        for version_id, stages in metadata.get('VersionIdsToStages', {}).items():
            if 'AWSPENDING' in stages and 'AWSCURRENT' in stages:
                logger.info('Removing stuck AWSPENDING from version: %s' % version_id)
                client.update_secret_version_stage(
                    SecretId=BACKUP_SECRET_ID,
                    VersionStage='AWSPENDING',
                    RemoveFromVersionId=version_id
                )

        # ── 2. Cancel any in-progress rotation ────────────────────────────
        try:
            client.cancel_rotate_secret(SecretId=BACKUP_SECRET_ID)
            logger.info('Cancelled existing in-progress rotation')
        except client.exceptions.InvalidRequestException as e:
            logger.info('Nothing to cancel: %s' % e)

        # ── 3. Trigger rotation with a unique UUID ClientRequestToken ─────
        # AWS throttles rotate_secret to once per 24 hours if the same token
        # is reused. A fresh UUID per invocation bypasses this throttle,
        # enabling rotation every 5 minutes (or faster).
        new_token = str(uuid.uuid4())

        response = client.rotate_secret(
            SecretId=BACKUP_SECRET_ID,
            ClientRequestToken=new_token
        )

        logger.info(
            'Backup rotation triggered. VersionId: %s, Token: %s'
            % (response.get('VersionId'), new_token)
        )
```

```
        return {
            'statusCode': 200,
            'body': 'Rotation triggered for %s' % BACKUP_SECRET_ID
        }

    except Exception as e:
        logger.error('Failed to trigger backup rotation: %s' % e)
        raise e
```

> A primary trigger Lambda is also needed — create a second Lambda called trigger-rotation-primary with identical code, but change BACKUP_SECRET_ID to 'db-rotation-new-secret'. Both trigger Lambdas use the same pattern.

STEP 3.2 Configure the Two Staggered EventBridge Rules

Rule A — Primary Secret (cron 0/5)

58. EventBridge → Rules → Create rule
59. Name: rotate-primary-secret, Rule type: Schedule
60. Schedule: cron(0/5 * * * ? *) — fires at 0, 5, 10, 15, 20... minutes past every hour
61. Target: Lambda → trigger-rotation-primary
62. Input: Constant JSON: {}
63. Create rule

Rule B — Backup Secret (cron 2/5 — offset by 2 minutes)

64. Create rule → Name: rotate-backup-secret
65. Schedule: cron(2/5 * * * ? *) — fires at 2, 7, 12, 17, 22... minutes
66. Target: Lambda → trigger-rotation-backup
67. Input: Constant JSON: {}
68. Create rule

Stagger Timing — What Happens Each Cycle

Time	Primary Secret	Backup Secret	App Can Use
0:00	Rotation STARTS	Stable — fully valid	Backup only
0:01	createSecret — new pwd stored	Stable	Backup
0:02	setSecret — pwd changed in DB	Stable	Backup (primary invalid)

0:03	testSecret — new pwd verified	Stable	Backup
0:04	finishSecret — primary live	Stable	Both valid
2:00	Stable — fully valid	Rotation STARTS	Primary only
2:04	Stable	finishSecret — backup live	Both valid
5:00	Rotation STARTS again	Stable	Backup only

> The 2-minute stagger guarantees at least one secret is always stable. The application failover layer (Phase 4) handles the transparent switch between primary and backup during the brief rotation window.

PHASE 4

Application-Layer Failover Manager

The application layer catches dropped connections mid-query, retries transparently on the backup engine, and rehydrates the primary engine in the background. This is what makes the rotation invisible to end users.

STEP 4.1 Framework Choice — SQLAlchemy vs Raw pymysql

Both approaches work with this architecture. Choose based on your application's existing stack.

Aspect	SQLAlchemy + NullPool	Raw pymysql
Connection pooling	NullPool — no pooling, stale connections never reused	Manual — you manage connect/close
Transaction handling	engine.begin() context manager — auto rollback	conn.begin() / conn.commit() / conn.rollback()
Error types	OperationalError, DisconnectionError (SQLAlchemy wrappers)	pymysql.OperationalError (raw MySQL codes)
Error code access	exc.orig.args[0]	exc.args[0] directly
Setup complexity	Higher — requires sqlalchemy install	Lower — single dependency
Best for	Existing SQLAlchemy projects, ORM usage	Lightweight scripts, microservices, new projects
Failover pattern	Replace engine object on credential drop	Replace connection object on credential drop

STEP 4.2 SQLAlchemy Implementation Pattern

Core Pattern — What the Code Must Do

- Initialize two engines at startup: primary_engine (app-user) and backup_engine (app-user-backup)
- On OperationalError with code 1045, 2006, or 2013 — mark primary as stale, retry on backup
- Apply exponential jitter sleep before retry — prevents 50,000 threads hammering the DB simultaneously
- Use double-checked locking for credential refresh — only one thread hits Secrets Manager even if 50,000 fail
- Trigger background thread to rehydrate primary engine after failover

```python
# SQLAlchemy approach — key imports and engine setup
from sqlalchemy import create_engine, text
from sqlalchemy.exc import OperationalError, DisconnectionError
from sqlalchemy.pool import NullPool

# NullPool is critical — prevents stale connections from being reused after rotation
def build_engine(creds):
    url = (f"mysql+pymysql://{creds['username']}:{creds['password']}"
           f"@{creds['host']}:{creds.get('port',3306)}/{creds.get('dbname','')}")
    return create_engine(url, poolclass=NullPool, connect_args={'connect_timeout': 5})

# Catch credential drop errors — these codes indicate a rotation event
# 1045 = Access denied  |  2006 = Server gone away  |  2013 = Lost connection
CREDENTIAL_DROP_CODES = {1045, 1044, 2006, 2013}

# On OperationalError, extract MySQL code:
err_code = exc.orig.args[0] if hasattr(exc, 'orig') else None

# Exponential jitter — staggers 50,000 threads across 0.5-3 seconds
# Prevents thundering herd on DB and Secrets Manager API
import random, time
jitter = random.uniform(0.5, 3.0) * attempt
time.sleep(jitter)

# Double-checked locking — only ONE thread fetches new credentials
if time.time() - self._last_refresh > self._cooldown:      # first check, no lock
    with self._lock:                                        # acquire lock
        if time.time() - self._last_refresh > self._cooldown:  # second check
            new_creds = fetch_secret(primary_secret_id)    # only one thread here
            self.primary_engine = build_engine(new_creds)
            self._last_refresh = time.time()
```

STEP 4.3 Raw pymysql Implementation Pattern

Core Pattern — pymysql Version

```python
# pymysql approach — key imports and connection setup
import pymysql
import boto3, json, threading, time, random

# Build a raw pymysql connection from secret credentials
def build_connection(creds):
    return pymysql.connect(
        host=creds['host'],
        user=creds['username'],
        password=creds['password'],
        port=int(creds.get('port', 3306)),
        database=creds.get('dbname'),
        connect_timeout=5,
        autocommit=False
    )
```

```
# pymysql error codes are accessed directly — no .orig wrapper
except pymysql.OperationalError as e:
    err_code = e.args[0]   # direct access — no exc.orig needed
    if err_code in {1045, 1044, 2006, 2013}:
        # failover logic here

# Transaction pattern — manual begin/commit/rollback
conn = build_connection(primary_creds)
try:
    conn.begin()
    with conn.cursor() as cur:
        cur.execute('INSERT INTO orders VALUES (%s, %s)', (user_id, amount))
        cur.execute('UPDATE inventory SET qty = qty - 1 WHERE sku = %s', (sku,))
    conn.commit()
except pymysql.OperationalError as e:
    conn.rollback()
    if e.args[0] in {1045, 1044, 2006, 2013}:
        # retry on backup connection
        backup_conn = build_connection(backup_creds)
        # ... retry transaction
finally:
    conn.close()
```

SQLAlchemy vs pymysql — Decision Summary

Your Situation	Recommended Approach
You already use SQLAlchemy in your project	SQLAlchemy + NullPool — consistent with existing code
You're building a new lightweight service	Raw pymysql — simpler, fewer dependencies
You use Django ORM or Flask-SQLAlchemy	SQLAlchemy — integrate with existing engine factory
You're writing ETL scripts or data pipelines	Raw pymysql — direct control, easier to debug
You need ORM model support (User.query, etc.)	SQLAlchemy — ORM requires it
You need maximum performance and control	Raw pymysql — no abstraction overhead

> Regardless of which framework you use, the failover pattern is identical: catch credential drop error codes, apply jitter, retry on backup, trigger background rehydration of primary. Only the syntax differs.

PHASE 5

Validation & Chaos Testing

The stress test proves zero-downtime under violent rotation conditions. It spawns 50 concurrent threads executing multi-step transactions with artificial latency delays — the exact scenario that causes failures in standard rotation setups.

STEP 5.1 transaction_stress_test.py — Complete Code

```
# transaction_stress_test.py
# Multi-threaded chaos stress test
# Run this while EventBridge is actively rotating both secrets
# Pass criteria: 0 hard failures, 100% data integrity

import uuid, time, random, threading, logging
from concurrent.futures import ThreadPoolExecutor, as_completed

# Use whichever connection manager you implemented in Phase 4
# Replace this import with your actual implementation
from db_connection_manager import DBConnectionManager

logging.basicConfig(level=logging.INFO,
    format='%(asctime)s [%(threadName)s] %(levelname)s: %(message)s')
logger = logging.getLogger(__name__)

# ── Test Configuration ──────────────────────────────────────────────
NUM_THREADS           = 50      # Concurrent simulated users
OPS_PER_THREAD        = 100     # Transactions per thread (5,000 total)
STRIPE_DELAY_MIN      = 0.05    # Min simulated external API latency (seconds)
STRIPE_DELAY_MAX      = 0.20    # Max simulated external API latency
LONG_QUERY_PROBABILITY= 0.10    # 10% of ops are long-running (chaos scenario)
LONG_QUERY_SLEEP      = 3.0     # Long query sleep duration

# ── Audit Tracking ──────────────────────────────────────────────────
audit_lock     = threading.Lock()
inserted_uuids = set()    # All UUIDs we intended to insert
committed_uuids= set()    # All UUIDs successfully committed
hard_failures  = []       # Unrecovered errors


def setup_test_table(db):
    """Create audit table for tracking inserts."""
    db.execute('''
        CREATE TABLE IF NOT EXISTS stress_test_audit (
            id         BIGINT AUTO_INCREMENT PRIMARY KEY,
            test_uuid  VARCHAR(36) UNIQUE NOT NULL,
            thread_id  INT,
            op_num     INT,
            status     VARCHAR(20) DEFAULT 'PENDING',
            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
        )
```

```
    ''')


def run_transaction(db, thread_id, op_num):
    """
    Execute a 3-step transaction with artificial latency.
    This simulates: INSERT record → wait for external API → UPDATE status
    If this transaction is killed mid-sleep by a rotation event,
    the DBConnectionManager rescues it on the backup engine.
    """
    test_uuid = str(uuid.uuid4())

    # Register as intended before attempting
    with audit_lock:
        inserted_uuids.add(test_uuid)

    is_long = random.random() < LONG_QUERY_PROBABILITY
    delay   = LONG_QUERY_SLEEP if is_long else random.uniform(STRIPE_DELAY_MIN,
STRIPE_DELAY_MAX)

    try:
        with db.transaction() as conn:
            # Step 1: INSERT
            conn.execute(
                'INSERT INTO stress_test_audit (test_uuid, thread_id, op_num) VALUES
(%s, %s, %s)',
                (test_uuid, thread_id, op_num)
            )
            # Step 2: Simulate external latency
            # This is where the chaos happens — rotation fires while we sleep
            time.sleep(delay)
            # Step 3: UPDATE to mark complete
            conn.execute(
                "UPDATE stress_test_audit SET status='COMPLETED' WHERE test_uuid=%s",
                (test_uuid,)
            )

        with audit_lock:
            committed_uuids.add(test_uuid)

    except Exception as e:
        logger.error('[T%d Op%d] HARD FAILURE: %s', thread_id, op_num, e)
        with audit_lock:

hard_failures.append({'thread':thread_id,'op':op_num,'uuid':test_uuid,'err':str(e)})


def verify_integrity(db):
    """Verify every committed UUID is physically on disk."""
    rows     = db.execute('SELECT test_uuid FROM stress_test_audit')
    on_disk  = {r[0] for r in rows}
    missing  = committed_uuids - on_disk
    pct      = len(committed_uuids) / max(len(inserted_uuids), 1) * 100
    return len(missing), pct


def run_stress_test():
```

```
    db = DBConnectionManager(
        primary_secret_id = 'db-rotation-new-secret',
        backup_secret_id  = 'db-rotation-backup-secret',
        region_name       = '<YOUR_REGION>'
    )
    setup_test_table(db)

    logger.info('Starting stress test: %d threads x %d ops = %d total',
                NUM_THREADS, OPS_PER_THREAD, NUM_THREADS * OPS_PER_THREAD)

    start = time.time()
    with ThreadPoolExecutor(max_workers=NUM_THREADS) as ex:
        futures = [
            ex.submit(lambda tid=t: [run_transaction(db, tid, op) for op in
range(OPS_PER_THREAD)])
            for t in range(NUM_THREADS)
        ]
        for f in as_completed(futures):
            try: f.result()
            except Exception as e: logger.error('Thread error: %s', e)

    elapsed = time.time() - start
    missing, pct = verify_integrity(db)

    logger.info('—' * 55)
    logger.info('STRESS TEST RESULTS')
    logger.info('—' * 55)
    logger.info('Duration          : %.1fs', elapsed)
    logger.info('Total operations  : %d', NUM_THREADS * OPS_PER_THREAD)
    logger.info('Hard failures     : %d', len(hard_failures))
    logger.info('Data integrity    : %.2f%%', pct)
    logger.info('Missing from disk : %d', missing)
    logger.info('—' * 55)
    passed = len(hard_failures) == 0 and missing == 0 and pct >= 100.0
    logger.info('RESULT: %s', 'PASS' if passed else 'FAIL')
    return passed


if __name__ == '__main__':
    import sys
    sys.exit(0 if run_stress_test() else 1)
```

STEP 5.2 Pass Criteria and Expected Output

Test	What Is Measured	Pass Condition
Hard failures	Unrecovered exceptions across all 50 threads	= 0
Data integrity	UUID audit — committed inserts vs rows on disk	= 100.00%
Thundering herd	Secrets Manager API calls during a rotation event	1-3 calls, not 50,000

Long query rescue	Queries killed mid-sleep rescued on backup	100% rescued
Transaction safety	Multi-step transactions killed mid-flight	Rolled back and re-executed on backup

Expected Log Output During a Rotation Event

```
# Rotation event detected — transparent failover to backup:
[Thread-12] WARNING: Credential drop (code 2006) — failing over to backup
[Thread-12] DEBUG: Jitter sleep: 1.43s
[Thread-12] INFO: Transaction rescued on backup engine (attempt 1)
[rehydration] INFO: Rehydrating primary engine from Secrets Manager
[rehydration] INFO: Primary rehydrated. User: app-user

# Final test output — passing run:
────────────────────────────────────────────────────────
STRESS TEST RESULTS
────────────────────────────────────────────────────────
Duration          : 312.4s
Total operations  : 5000
Hard failures     : 0
Data integrity    : 100.00%
Missing from disk : 0
────────────────────────────────────────────────────────
RESULT: PASS
```

How to Run the Test

69. Install dependencies: pip install sqlalchemy pymysql boto3
70. Ensure your test machine is inside the VPC (or has network access to RDS)
71. Verify both EventBridge rules are enabled and firing
72. Run: python transaction_stress_test.py
73. Watch logs — you should see failover events during rotation minutes (0, 2, 5, 7...)
74. At completion, verify RESULT: PASS

Troubleshooting

Symptom	Cause	Fix
Lambda never logs anything (storedBytes: 0 in CloudWatch)	No Secrets Manager VPC endpoint	Create com.amazonaws.<region>.secretsmanager Interface endpoint with rotation-sg and private DNS enabled
Lambda times out after 'Found credentials'	No RDS VPC endpoint OR zero egress on security group	Create com.amazonaws.<region>.rds endpoint; verify rotation-sg has port 443 and 3306 outbound
setSecret: Unable to log into database	Secret out of sync with DB password	The self-healing modification (Phase 2.3) fixes this — verify you removed the AWSCURRENT validation block
A previous rotation is not complete	Stuck AWSPENDING version	The trigger Lambda cleanup code removes this automatically. Can also run cancel-rotate-secret manually
Both engines fail simultaneously	Both secrets rotating at same time — stagger misconfigured	Verify cron(0/5) vs cron(2/5). The offset must be at least 2 minutes
Thundering herd not contained	Jitter not applied or cooldown window too short	Verify jitter sleep is applied before backup retry; increase _refresh_cooldown to 60s
Primary never rehydrates after failover	Background thread not starting or rehydration failing	Check CloudWatch logs for rehydration thread errors; verify IAM role has secretsmanager:GetSecretValue
EventBridge rule not triggering Lambda	Lambda resource-based policy missing	Verify Lambda resource policy includes secretsmanager.amazonaws.com and events.amazonaws.com as principals
CLI hangs from bastion EC2	Bastion security group missing port 443 outbound	Add port 443 outbound to bastion-sg
SSM Session Manager shows Offline	SSM VPC endpoints not created	Create ssm, ssmmessages, ec2messages Interface endpoints

Quick Reference

All Resources Created

Resource	Name	Purpose
VPC	rotation-vpc	Private network — all components
Security Group	rotation-sg	Self-referencing — Lambda, RDS, endpoints
VPC Endpoint (x5)	sm, rds, ssm, ssmmessages, ec2messages	Private AWS service access — no NAT needed
RDS Instance	<DB_NAME>	MySQL 8.0 — private subnet
DB User	app-user	Primary rotating user
DB User	app-user-backup	Backup user — rotates on offset schedule
Secret	db-rotation-master	Admin credentials — never rotates
Secret	db-rotation-new-secret	Primary credentials — cron(0/5)
Secret	db-rotation-backup-secret	Backup credentials — cron(2/5)
Lambda	SecretsManager-rotation-<DB_NAME>	Shared rotation Lambda — handles both secrets
Lambda	trigger-rotation-primary	Cleanup + trigger for primary secret
Lambda	trigger-rotation-backup	Cleanup + trigger for backup secret (deploy_backup_trigger.py)
EventBridge Rule	rotate-primary-secret	cron(0/5) — fires at 0, 5, 10, 15... min
EventBridge Rule	rotate-backup-secret	cron(2/5) — fires at 2, 7, 12, 17... min

MySQL Error Code Reference

Code	Meaning	Application Action
1045	Access denied — wrong password	Credential was rotated → failover to backup
1044	Database access denied	Credential was rotated → failover to backup
2006	MySQL server has gone away	Connection severed by rotation → failover
2013	Lost connection to MySQL	Connection lost mid-query → failover
Other codes	Non-credential error	Re-raise — do not failover

Stagger Schedule — At a Glance

Minute	Primary	Backup	Safe to Route To
0:00 — 0:04	ROTATING	Stable	Backup engine only
0:05 — 1:59	Stable	Stable	Either engine
2:00 — 2:04	Stable	ROTATING	Primary engine only
2:05 — 4:59	Stable	Stable	Either engine
5:00 — 5:04	ROTATING	Stable	Backup engine only